Technical Requirements Document

Project: CareerGuid AI / SkillSight

Document Type: Technical Requirements Document
Prepared From: Final Product Requirement Document
Role Perspective: Senior Software Architect

---

1. Project Overview

CareerGuid AI is an AI-powered career guidance platform that helps students, freshers, and job seekers become job-ready.

The system allows users to upload their resume first, automatically extracts profile details, lets users edit or complete missing information, analyzes skill gaps against a target job role, generates a personalized RAG-based roadmap, sends weekly reminders if users are not progressing, and helps users build ATS-friendly resumes.

The platform will be designed as a scalable full-stack web application using:

- React.js frontend
- Node.js backend
- Python AI service
- Supabase PostgreSQL database
- Supabase Storage
- Supabase Auth
- Supabase pgvector for RAG
- Redis for cache, rate limiting, queues, and temporary data
- Docker-based deployment
- Nginx reverse proxy / load balancer

---

2. High-Level Technical Architecture

```
User
  v
React Frontend
  v
Nginx / Load Balancer
  v
Backend API Server - Node.js / Express.js
  v
Services Layer
```

```
  |-- Auth Service
  |-- Onboarding Service
  |-- Resume Upload Service
  |-- Resume Parsing Integration Service
  |-- Skill Gap Service
  |-- Roadmap Service
  |-- ATS Resume Builder Service
  |-- Notification Service
  `-- Admin Service

AI Layer
  |-- Python FastAPI AI Service
  |-- Resume Parser
  |-- Skill Extractor
  |-- RAG Pipeline
  |-- Embedding Generator
  `-- GenAI Agent

Data Layer
  |-- Supabase PostgreSQL
  |-- Supabase Storage
  |-- Supabase pgvector
  `-- Redis

Background Layer
  |-- BullMQ / Redis Queue
  |-- Email Worker
  |-- Resume Parsing Worker
  |-- AI Generation Worker
  `-- Weekly Reminder Scheduler
```

---

3. Frontend Technical Requirements

3.1 Frontend Stack

Area	Technology
Framework	React.js
Language	TypeScript
Build Tool	Vite
Routing	React Router
State Management	Context API / Zustand
API Communication	Axios / Fetch
Form Handling	React Hook Form

Area	Technology
Validation	Zod
Styling	CSS Modules / Plain CSS
Charts	Recharts
PDF Preview	Browser iframe / PDF viewer
Authentication State	Supabase Auth Client + JWT session

3.2 Frontend Modules

1. Landing Page

Purpose:

- Explain product value
- Show features
- Provide login/register CTA

Pages:

```
/
/login
/register
/forgot-password
```

2. Authentication UI

Features:

- Register
- Login
- Email OTP verification
- Forgot password
- Reset password

Frontend should handle:

- Form validation
- Error messages
- Loading states
- Token/session storage using Supabase client
- Redirect after login

3. Resume-First Smart Onboarding UI

Flow:

```
User logs in first time
    v
Resume upload screen
```

```
    v
Upload PDF/DOCX resume
    v
System extracts data
    v
Auto-filled profile form appears
    v
User edits incorrect fields
    v
User fills missing fields
    v
Final submit
    v
Dashboard
```

Frontend requirements:

- Resume upload component
- File validation before upload
- Auto-filled profile form
- Editable fields
- Missing field indicators
- Field source label: Auto-filled / Manual
- Final review screen
- Submit profile button

Fields:

```
Full name
Email
Phone number
Current city
Education
Work experience
Skills
Projects
LinkedIn URL
GitHub URL
Portfolio URL
Target job role
Preferred location
Work preference
Expected salary
Notice period
Career goal
```

4. Dashboard UI

Dashboard should show:

- Profile completion percentage
- Resume match score
- Target role
- Current skills
- Missing skills
- Roadmap progress percentage
- Pending roadmap tasks
- Completed tasks
- ATS resume score
- Recent AI suggestions
- Reminder status

5. Skill Gap Analysis UI

Frontend should allow:

- Target job role selection
- Skill gap analysis request
- Current skills display
- Missing skills display
- Match score display
- Priority learning order

6. RAG Roadmap UI

Frontend should display:

- Week-wise roadmap
- Task list inside each week
- Task status: pending/completed
- Mark task as complete
- Progress percentage
- Upcoming due tasks
- Reminder notice if behind schedule

7. ATS Resume Builder UI

Frontend should support:

- Generate resume
- Preview resume
- Edit generated sections
- Add/remove skills
- Improve project descriptions
- Download PDF

- Download DOCX

8. Admin Panel UI

Admin can manage:

- Users
- Job roles
- Required skills
- Roadmap knowledge base
- AI prompt templates
- Reminder logs
- Failed AI jobs

---

4. Backend Technical Requirements

4.1 Backend Stack

Area	Technology
Runtime	Node.js
Framework	Express.js
Language	TypeScript
Authentication	Supabase Auth JWT verification
Validation	Zod / Joi
ORM / DB Client	Supabase JS Client / Prisma with PostgreSQL
Queue	BullMQ
Cache	Redis
File Upload	Multer / Busboy
Email	Resend / SendGrid / SMTP provider
PDF Generation	Puppeteer / PDFKit
DOCX Generation	docx npm package
Logging	Winston / Pino
API Documentation	Swagger / OpenAPI
Testing	Jest + Supertest

4.2 Backend Architecture Style

Recommended architecture: **Modular Monolith for MVP**

Reason:

- Easier to build and maintain initially
- Faster development for student/fresher-level project
- Clear separation of modules

- Can later split into microservices if traffic grows

Backend folders:

```
backend/
 |-- src/
 |   |-- config/
 |   |-- middlewares/
 |   |-- modules/
 |   |   |-- auth/
 |   |   |-- onboarding/
 |   |   |-- resume/
 |   |   |-- skill-gap/
 |   |   |-- roadmap/
 |   |   |-- resume-builder/
 |   |   |-- reminders/
 |   |   `-- admin/
 |   |-- services/
 |   |-- queues/
 |   |-- workers/
 |   |-- utils/
 |   |-- routes/
 |   `-- server.ts
```

---

5. Database Requirements

5.1 Database Stack

Area	Technology
Primary Database	Supabase PostgreSQL
File Storage	Supabase Storage
Vector Search	Supabase pgvector
Authentication Store	Supabase Auth
Temporary Data	Redis
Queue Storage	Redis / BullMQ

5.2 Why Supabase Instead of MongoDB

Supabase is selected because:

- It provides managed PostgreSQL
- It supports relational data strongly
- It includes built-in authentication
- It supports Row Level Security

- It provides file storage
- It supports pgvector for RAG
- It reduces backend complexity
- It is better for structured product data like users, profiles, resumes, roadmaps, tasks, and logs

5.3 Main Database Tables

users_profile

Stores extended user profile after onboarding.

```
users_profile
- id UUID PRIMARY KEY
- auth_user_id UUID REFERENCES auth.users(id)
- full_name TEXT
- email TEXT
- phone TEXT
- current_city TEXT
- education TEXT
- work_experience TEXT
- skills TEXT[]
- projects JSONB
- linkedin_url TEXT
- github_url TEXT
- portfolio_url TEXT
- target_job_role TEXT
- preferred_location TEXT
- work_preference TEXT
- expected_salary TEXT
- notice_period TEXT
- career_goal TEXT
- onboarding_completed BOOLEAN
- created_at TIMESTAMP
- updated_at TIMESTAMP
```

resumes

Stores uploaded resume metadata and extracted data.

```
resumes
- id UUID PRIMARY KEY
- user_id UUID REFERENCES users_profile(id)
- file_url TEXT
- file_name TEXT
- file_type TEXT
- file_size INTEGER
```

- extracted_text TEXT
- extracted_data JSONB
- extracted_skills TEXT[]
- parsing_status TEXT
- created_at TIMESTAMP

job_roles

Stores target job roles.

```
job_roles
- id UUID PRIMARY KEY
- role_name TEXT
- role_description TEXT
- category TEXT
- created_at TIMESTAMP
```

role_skills

Stores required skills for each job role.

```
role_skills
- id UUID PRIMARY KEY
- role_id UUID REFERENCES job_roles(id)
- skill_name TEXT
- priority TEXT
- skill_type TEXT
```

skill_analysis

Stores skill gap result.

```
skill_analysis
- id UUID PRIMARY KEY
- user_id UUID REFERENCES users_profile(id)
- role_id UUID REFERENCES job_roles(id)
- current_skills TEXT[]
- missing_skills TEXT[]
- recommended_skills TEXT[]
- match_score INTEGER
- analysis_result JSONB
- created_at TIMESTAMP
```

roadmaps

Stores AI-generated roadmap.

```
roadmaps
- id UUID PRIMARY KEY
```

```
- user_id UUID REFERENCES users_profile(id)
- role_id UUID REFERENCES job_roles(id)
- title TEXT
- description TEXT
- progress_percentage INTEGER
- generated_by TEXT
- created_at TIMESTAMP
- updated_at TIMESTAMP
```

roadmap_tasks

Stores week-wise roadmap tasks.

```
roadmap_tasks
- id UUID PRIMARY KEY
- roadmap_id UUID REFERENCES roadmaps(id)
- week_number INTEGER
- task_title TEXT
- task_description TEXT
- due_date DATE
- status TEXT
- completed_at TIMESTAMP
- created_at TIMESTAMP
```

reminder_logs

Stores weekly reminder history.

```
reminder_logs
- id UUID PRIMARY KEY
- user_id UUID REFERENCES users_profile(id)
- roadmap_id UUID REFERENCES roadmaps(id)
- week_number INTEGER
- reminder_type TEXT
- email_sent BOOLEAN
- sent_at TIMESTAMP
- created_at TIMESTAMP
```

generated_resumes

Stores AI-generated ATS resumes.

```
generated_resumes
- id UUID PRIMARY KEY
- user_id UUID REFERENCES users_profile(id)
- target_role TEXT
- resume_content JSONB
- ats_keywords TEXT[]
```

- pdf_url TEXT
- docx_url TEXT
- created_at TIMESTAMP

knowledge_base_documents

Stores RAG documents.

```
knowledge_base_documents
- id UUID PRIMARY KEY
- title TEXT
- category TEXT
- content TEXT
- metadata JSONB
- embedding VECTOR
- created_at TIMESTAMP
```

---

6. Authentication Requirements

6.1 Authentication Method

Recommended method: **Supabase Auth + JWT**

Flow:

```
User registers
   v
Supabase Auth creates account
   v
Email OTP / verification sent
   v
User verifies email
   v
Supabase issues JWT session
   v
Frontend stores session securely
   v
Backend verifies JWT on protected APIs
```

6.2 Auth Features

Required:

- Register
- Login
- Logout

- Email verification
- Forgot password
- Reset password
- JWT-based protected APIs
- Role-based access control
- Admin role support

6.3 Authorization

Roles:

```
guest
user
admin
ai_worker
```

Rules:

- Guest can only access public routes
- User can access only own profile, resumes, roadmaps, generated resumes
- Admin can manage roles, knowledge base, users, logs
- Worker can process jobs but should not expose public API

Supabase Row Level Security should be enabled for user-level data.

---

7. API Requirements

7.1 Auth APIs

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/logout
POST /api/auth/verify-email
POST /api/auth/forgot-password
POST /api/auth/reset-password
GET  /api/auth/me
```

7.2 Onboarding APIs

```
POST /api/onboarding/resume-upload
POST /api/onboarding/auto-fill
GET  /api/onboarding/profile
PUT  /api/onboarding/profile
POST /api/onboarding/complete
```

Purpose:

- Upload resume
- Extract data
- Auto-fill profile
- Save edited profile
- Mark onboarding complete

7.3 Resume APIs

```
POST /api/resumes/upload
GET  /api/resumes
GET  /api/resumes/:resumeId
DELETE /api/resumes/:resumeId
POST /api/resumes/:resumeId/parse
```

7.4 Skill Gap APIs

```
GET  /api/job-roles
GET  /api/job-roles/:roleId/skills
POST /api/skill-gap/analyze
GET  /api/skill-gap/:analysisId
GET  /api/users/me/skill-gap/latest
```

7.5 Roadmap APIs

```
POST /api/roadmap/generate
GET  /api/roadmap
GET  /api/roadmap/:roadmapId
POST /api/roadmap/tasks/:taskId/complete
POST /api/roadmap/tasks/:taskId/reopen
GET  /api/roadmap/:roadmapId/progress
```

7.6 Reminder APIs

```
POST /api/reminders/check-weekly
GET  /api/reminders/logs
GET  /api/reminders/logs/:userId
```

Note:

- `POST /api/reminders/check-weekly` should be protected and used only by scheduler/admin.
- Normal users should not trigger reminder checks.

7.7 ATS Resume Builder APIs

```
POST /api/resume-builder/generate
GET  /api/resume-builder/:resumeId
```

```
PUT  /api/resume-builder/:resumeId
GET  /api/resume-builder/:resumeId/preview
GET  /api/resume-builder/:resumeId/download/pdf
GET  /api/resume-builder/:resumeId/download/docx
```

7.8 Admin APIs

```
GET  /api/admin/users
GET  /api/admin/users/:userId
POST /api/admin/job-roles
PUT  /api/admin/job-roles/:roleId
DELETE /api/admin/job-roles/:roleId
POST /api/admin/knowledge-base
GET  /api/admin/knowledge-base
DELETE /api/admin/knowledge-base/:documentId
GET  /api/admin/analytics
GET  /api/admin/logs
```

7.9 Health Check APIs

```
GET /api/health
GET /api/health/db
GET /api/health/redis
GET /api/health/ai-service
```

---

8. AI Model and AI Tools Requirements

8.1 AI Service Stack

Area	Technology
AI Service Framework	Python FastAPI
Resume Text Extraction	pdfplumber, PyMuPDF, python-docx
NLP Processing	spaCy / regex / custom skill dictionary
Embeddings	OpenAI-compatible embedding model / Sentence Transformers
Vector Database	Supabase pgvector
RAG Framework	LangChain / LlamaIndex
LLM	OpenAI-compatible LLM / Gemini-compatible LLM
API Communication	REST API
Response Validation	Pydantic schemas

8.2 AI Service Responsibilities

The AI service should handle:

- Resume text extraction
- Profile detail extraction
- Skill extraction
- Skill normalization
- Skill gap reasoning
- RAG document retrieval
- Roadmap generation
- ATS resume content generation
- Project description improvement
- Resume summary generation

8.3 Resume Parsing Pipeline

```
Resume uploaded
   v
File stored in Supabase Storage
   v
Parsing job added to queue
   v
AI service downloads file
   v
Text extracted from PDF/DOCX
   v
Name, email, phone, education, skills, projects extracted
   v
Structured JSON returned
   v
Backend saves extracted data
   v
Frontend shows auto-filled form
```

Expected output:

```
{
  "fullName": "Nitin Kumar",
  "email": "nitin@example.com",
  "phone": "638xxxxxxx",
  "education": "B.Tech CSE",
  "skills": ["Java", "Node.js", "SQL"],
  "projects": [
    {
      "name": "Book Sharing Application",
      "description": "A web app for sharing books"
    }
```

```
  ],
  "linkedinUrl": "",
  "githubUrl": ""
}
```

8.4 Skill Gap Pipeline

```
User profile skills
   v
Target role required skills
   v
Normalize skills
   v
Compare exact matches
   v
Compare semantic matches using embeddings
   v
Calculate match score
   v
Identify missing skills
   v
Generate recommended learning order
```

8.5 RAG Roadmap Pipeline

```
User profile + missing skills + target role
   v
Create query embedding
   v
Search Supabase pgvector
   v
Retrieve role documents, skills, interview topics, resources
   v
Send context to LLM
   v
Generate week-wise roadmap
   v
Validate response JSON
   v
Save roadmap and tasks in Supabase
```

Roadmap output must be structured:

```
{
  "title": "Backend Developer Roadmap",
  "durationWeeks": 6,
  "weeks": [
```

```
    {
      "weekNumber": 1,
      "title": "JavaScript Basics",
      "tasks": [
        "Learn variables and functions",
        "Practice loops",
        "Understand ES6 features"
      ]
    }
  ]
}
```

8.6 ATS Resume Builder Pipeline

```
User profile + target role + skill gap
   v
Retrieve ATS keywords from knowledge base
   v
Generate resume summary
   v
Improve project descriptions
   v
Optimize skills section
   v
Generate resume JSON
   v
Render HTML template
   v
Generate PDF/DOCX
   v
Store file in Supabase Storage
   v
Return download URL
```

---

9. Redis and Queue Requirements

9.1 Redis Usage

Redis will be used for:

- Rate limiting
- OTP temporary storage, if custom OTP is used
- Forgot password temporary token, if custom token is used
- Cache AI responses

- Cache roadmap generation result
- BullMQ queue backend
- Temporary session metadata

9.2 Rate Limit Requirements

Important requirement:

The system should support **minimum 100 successful user logins per minute system-wide**.

Rate limiting should not block legitimate login capacity.

Recommended rules:

API	Limit
Successful login throughput	Minimum 100 users/minute system-wide
Failed login attempts	5 failed attempts/minute per IP + email
Send OTP	3 requests/10 minutes per email
Forgot Password	3 requests/15 minutes per email
Resume Upload	10 uploads/hour per user
AI Roadmap Generate	20 requests/day per user
ATS Resume Generate	10 requests/day per user

Reason:

- Login system should handle normal traffic
- Brute force attacks should still be blocked
- OTP and AI APIs are cost-sensitive, so stricter limits are needed

9.3 Queue Jobs

Required queues:

```
emailQueue
resumeParsingQueue
roadmapGenerationQueue
resumeBuilderQueue
weeklyReminderQueue
```

9.4 Worker Responsibilities

Email Worker

- Send OTP email
- Send welcome email
- Send forgot password email
- Send weekly roadmap reminder email

Resume Parsing Worker

- Process resume parsing jobs
- Retry failed parsing
- Save extracted data

Roadmap Worker

- Generate RAG-based roadmap
- Save roadmap tasks

Reminder Worker

- Check pending weekly tasks
- Create reminder emails
- Save reminder logs

---

10. Deployment Requirements

10.1 Deployment Setup

Recommended deployment for MVP:

```
Frontend: Vercel / Netlify / Docker Nginx
Backend: Docker container on VPS / Render / Railway / AWS EC2
AI Service: Docker container on VPS / Render / AWS EC2
Database: Supabase Cloud
Storage: Supabase Storage
Redis: Upstash Redis / Redis Docker container
Reverse Proxy: Nginx
CI/CD: GitHub Actions
```

10.2 Docker Containers

Required containers:

```
frontend-container
backend-container
ai-service-container
worker-container
scheduler-container
redis-container
nginx-container
```

Supabase will be managed externally, so no MongoDB container is required.

10.3 Docker Compose Services

```
frontend
backend
ai-service
worker
scheduler
redis
nginx
```

10.4 Environment Variables

Backend:

```
NODE_ENV=production
PORT=5000
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
SUPABASE_ANON_KEY=
JWT_SECRET=
REDIS_URL=
AI_SERVICE_URL=
EMAIL_PROVIDER_API_KEY=
FRONTEND_URL=
```

AI Service:

```
AI_SERVICE_PORT=8000
SUPABASE_URL=
SUPABASE_SERVICE_ROLE_KEY=
LLM_API_KEY=
EMBEDDING_API_KEY=
```

10.5 CI/CD Pipeline

Recommended GitHub Actions flow:

```
Push to main branch
   v
Run lint
   v
Run tests
   v
Build frontend
   v
Build backend Docker image
   v
Build AI service Docker image
```

```
   v
Deploy to server
   v
Run health checks
```

---

11. Security Requirements

11.1 Authentication Security

- Use Supabase Auth
- Verify JWT in backend middleware
- Use refresh token flow from Supabase
- Enable email verification
- Enable password reset
- Use secure password policies

11.2 Authorization Security

- Enable Supabase Row Level Security
- User can access only own records
- Admin-only APIs must check admin role
- Service role key must never be exposed to frontend

11.3 API Security

- Use HTTPS in production
- Use Helmet security headers
- Configure CORS for allowed frontend domain only
- Validate every request body using Zod/Joi
- Sanitize user input
- Use rate limiting with Redis
- Log suspicious activities

11.4 File Upload Security

- Allow only PDF and DOCX
- Max file size: 5 MB for MVP
- Validate MIME type
- Reject executable files
- Store files in private Supabase bucket
- Generate signed URLs for access
- Optional: virus scanning in production

11.5 AI Security

- Do not pass secrets to LLM
- Remove sensitive personal data where not required
- Use prompt injection protection in RAG
- Limit user-provided document influence
- Validate AI output using JSON schema
- Log AI failures safely
- Avoid storing unnecessary sensitive AI prompts

11.6 Redis Security

- Redis should not be publicly exposed
- Use Redis password/TLS in production
- Use key expiry for temporary values
- Keep rate-limit keys short-lived

11.7 Secret Management

- Use `.env` locally
- Use platform secrets in production
- Never commit API keys
- Rotate keys periodically
- Use different keys for dev and production

---

12. Performance Requirements

12.1 API Performance

Requirement	Target
Normal API response time	Under 500ms
Dashboard load time	Under 2 seconds
Login capacity	Minimum 100 successful logins/minute
Resume upload response	Under 3 seconds after upload accepted
Resume parsing	Async, usually within 30-60 seconds
Roadmap generation	Async, usually within 30-90 seconds
ATS resume generation	Async, usually within 30-90 seconds

12.2 Database Performance

Requirements:

- Index frequently queried columns
- Index `user_id`

- Index `role_id`
- Index `roadmap_id`
- Use pgvector index for embeddings
- Avoid storing very large raw files in database
- Store files in Supabase Storage

12.3 Caching Requirements

Use Redis cache for:

- Job role list
- Required skills list
- AI-generated roadmap result
- User dashboard summary
- Repeated RAG retrieval results

12.4 Scalability Requirements

The system should support:

- Horizontal scaling of backend containers
- Separate scaling of AI service
- Separate worker containers
- Queue-based async processing
- Supabase managed database scaling
- Redis-based distributed rate limiting

---

13. Third-Party Integrations

13.1 Supabase

Used for:

- PostgreSQL database
- Authentication
- Storage
- Row Level Security
- pgvector

Reason:

- Reduces infrastructure complexity
- Provides built-in auth and storage
- Supports relational schema
- Supports vector search

13.2 Email Provider

Options:

- Resend
- SendGrid
- Amazon SES
- SMTP provider

Used for:

- OTP email
- Welcome email
- Forgot password email
- Weekly roadmap reminder email
- Resume analysis completed email

Reason:

- Email delivery should be reliable
- Email should be handled asynchronously through queue

13.3 LLM Provider

Options:

- OpenAI-compatible API
- Gemini-compatible API
- Local open-source LLM in future

Used for:

- Roadmap generation
- Resume summary generation
- Project description improvement
- ATS keyword suggestions
- Career assistant responses

Reason:

- Faster development
- Better natural language generation
- Can be replaced later using provider abstraction

13.4 Embedding Provider

Options:

- OpenAI-compatible embedding API
- Sentence Transformers
- Other embedding models

Used for:

- RAG document embeddings
- Semantic skill matching
- Similarity search

Reason:

- Required for RAG and semantic retrieval

13.5 Monitoring Tools

Options:

- Sentry
- Logtail
- Grafana
- UptimeRobot

Used for:

- Error tracking
- API monitoring
- Worker failure alerts
- Uptime monitoring

---

14. Technical Decisions With Reasons

Decision	Selected Option	Reason
Frontend framework	React.js + TypeScript	Popular, scalable, component-based, good for dashboards
Build tool	Vite	Fast development and optimized builds
Backend framework	Node.js + Express.js	Simple, familiar, fast for REST APIs
Backend language	TypeScript	Better type safety and maintainability
Database	Supabase PostgreSQL	Structured data, built-in auth, storage, RLS, pgvector
File storage	Supabase Storage	Direct integration with Supabase auth and database
Vector database	Supabase pgvector	Avoids separate vector DB initially

Decision	Selected Option	Reason
Auth	Supabase Auth	Built-in email verification, JWT, password reset
Cache	Redis	Fast cache, rate limit, temporary data
Queue	BullMQ + Redis	Reliable background job processing
AI service	Python FastAPI	Python has strong AI/ML ecosystem
RAG framework	LangChain / LlamaIndex	Faster RAG pipeline development
Deployment	Docker + Nginx	Portable, scalable, production-friendly
Architecture	Modular monolith	Faster MVP, easier development, can scale later
Email sending	Queue-based worker	Avoids blocking main API
Resume parsing	Async worker	Resume parsing can be slow, should not block request
Weekly reminder	Scheduler + queue	Reliable background automation
PDF generation	Puppeteer / PDFKit	Good control over resume templates
DOCX generation	docx library	Allows editable resume export

---

15. Error Handling Requirements

The system should handle:

- Invalid login
- Expired OTP
- Too many failed login attempts
- Invalid resume file
- Resume parsing failure
- AI service timeout
- RAG generation failure
- Email delivery failure
- Supabase connection failure
- Redis connection failure

Error response format:

```
{
  "success": false,
  "message": "Resume parsing failed. Please try again.",
  "errorCode": "RESUME_PARSE_FAILED"
}
```

---

16. Logging and Monitoring Requirements

Backend should log:

- Login attempts
- Failed auth events
- Resume uploads
- Resume parsing status
- AI generation requests
- Roadmap generation
- Reminder emails sent
- API errors
- Worker failures

Do not log:

- Passwords
- OTP values
- Full JWT tokens
- Secret keys
- Sensitive personal data unnecessarily

---

17. Testing Requirements

17.1 Frontend Testing

- Form validation tests
- Component tests
- Auth flow tests
- Dashboard rendering tests
- Resume upload UI tests

17.2 Backend Testing

- API unit tests
- Auth middleware tests

- Rate limit tests
- Resume upload tests
- Skill gap tests
- Reminder logic tests

17.3 AI Testing

- Resume extraction test cases
- Skill extraction accuracy tests
- RAG response validation
- ATS resume output validation

17.4 Integration Testing

- Signup to onboarding flow
- Resume upload to auto-fill flow
- Skill gap to roadmap generation flow
- Roadmap pending task to reminder email flow
- Resume builder to PDF download flow

---

18. MVP Technical Scope

Included in MVP

- Supabase Auth
- Resume-first onboarding
- Resume upload
- Resume parsing
- Auto-filled profile
- Manual missing field completion
- Skill gap analysis
- Basic roadmap generation
- Dashboard
- Docker setup
- Redis rate limiting
- Supabase PostgreSQL schema

Not Included in MVP

- Recruiter portal
- Payment system
- Mobile app
- Advanced AI mock interview
- Advanced college dashboard

- Advanced analytics
- Full reminder automation, planned in later version
- Multi-tenant enterprise setup

---

19. Future Technical Enhancements

Future versions can include:

- Microservices migration
- Kubernetes deployment
- Advanced AI interview assistant
- Recruiter dashboard
- College analytics dashboard
- Payment gateway
- Mobile app
- Advanced ATS scoring
- AI-based job matching
- Multi-resume support
- Advanced monitoring with Grafana and Prometheus

---

20. Final Technical Summary

CareerGuid AI will be built as a scalable AI-powered career guidance platform using React.js, Node.js, Python FastAPI, Supabase PostgreSQL, Supabase Storage, Supabase pgvector, Redis, BullMQ, Docker, and Nginx.

The system will use Supabase Auth for authentication, Redis for rate limiting and queues, Python AI services for resume parsing and RAG, and Supabase pgvector for vector search.

The architecture is designed as a modular monolith for MVP with separate AI and worker services. This gives fast development speed while keeping the system scalable for future versions.

The most important technical flows are:

```
Resume-first onboarding
Skill gap analysis
RAG-based roadmap generation
Weekly reminder automation
ATS-friendly resume generation
Secure authentication
Docker-based deployment
```

This technical architecture is suitable for building a production-ready MVP and can later evolve into a more advanced career platform.